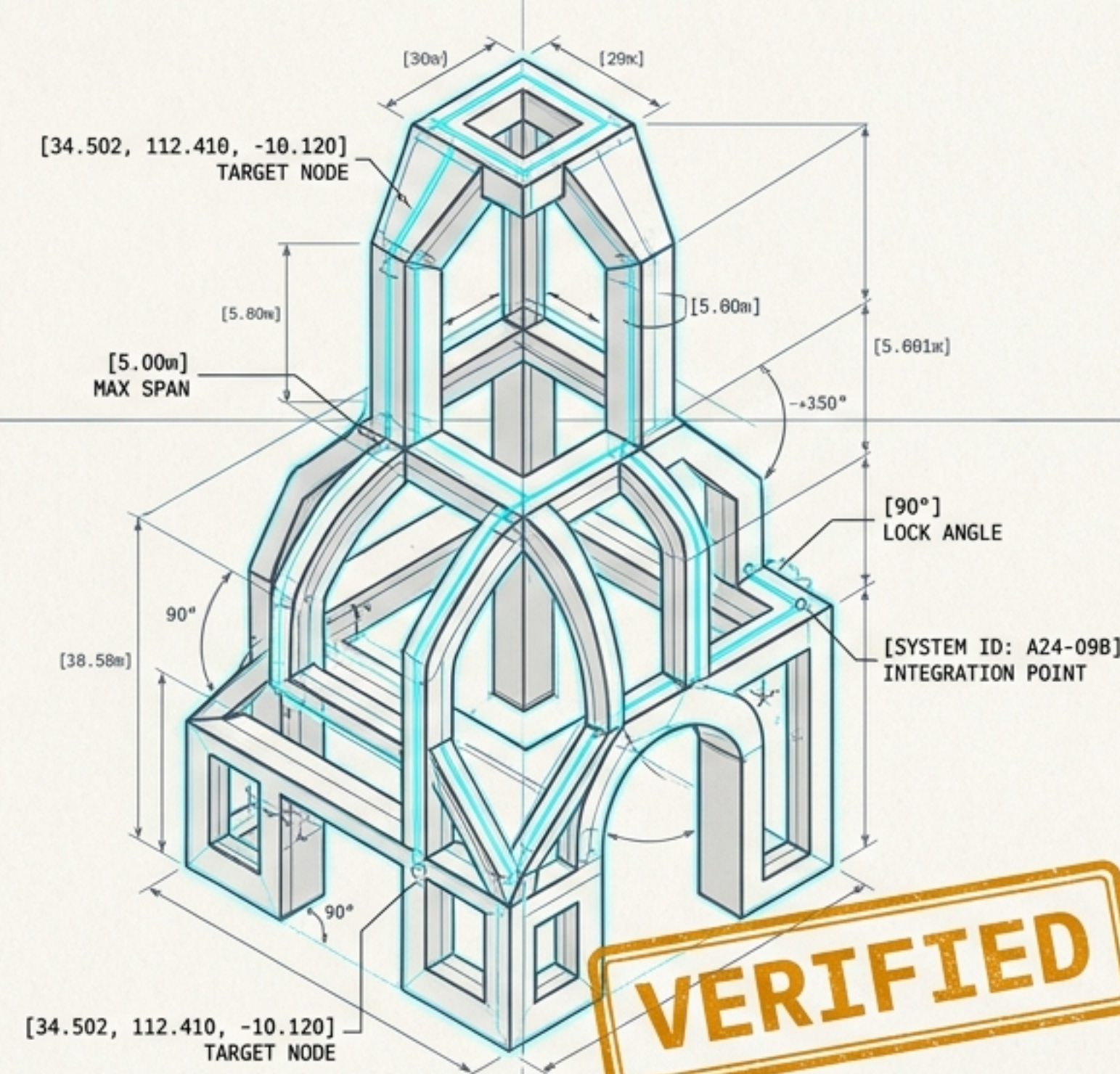
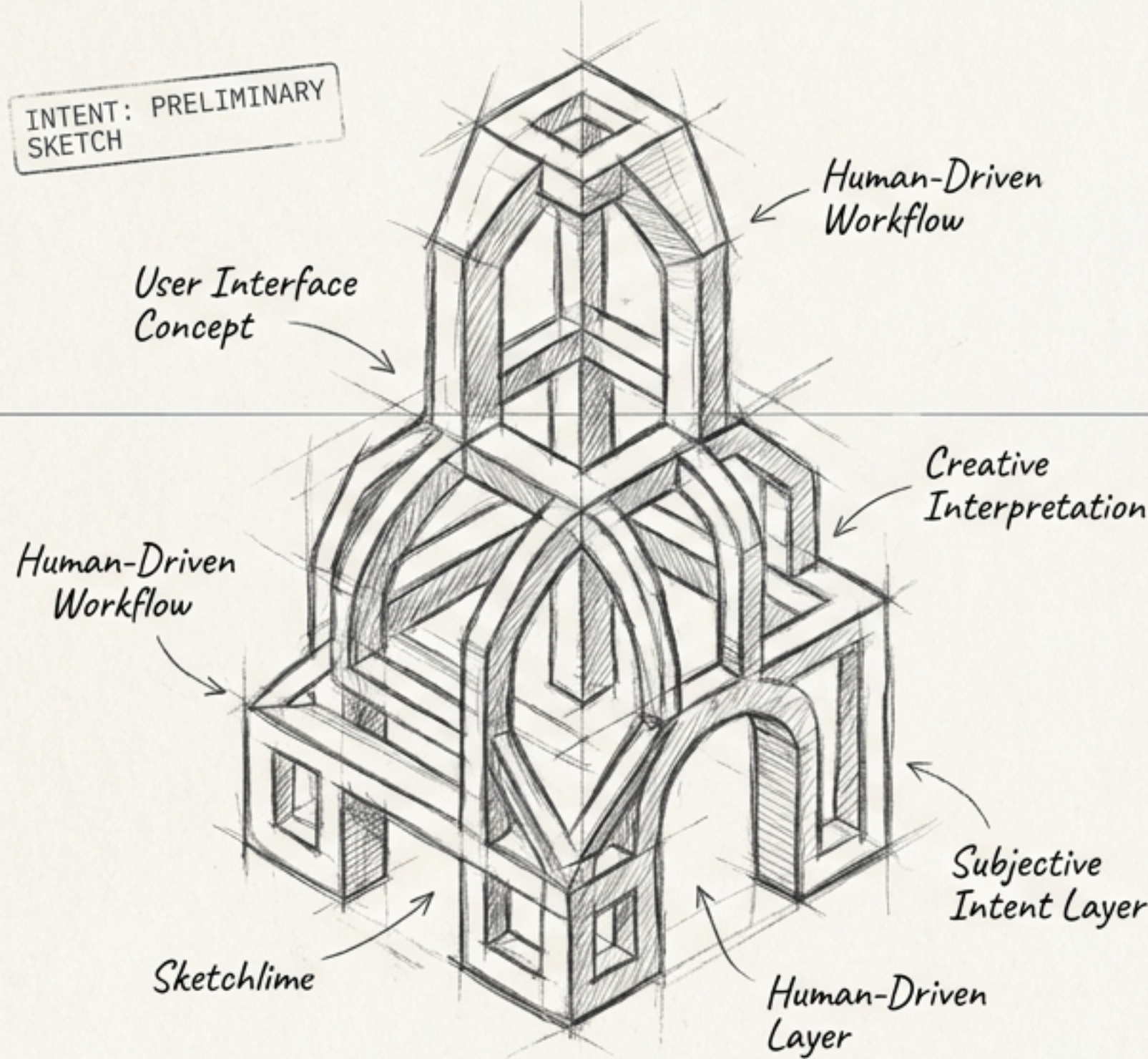


Authorship is Not Assurance

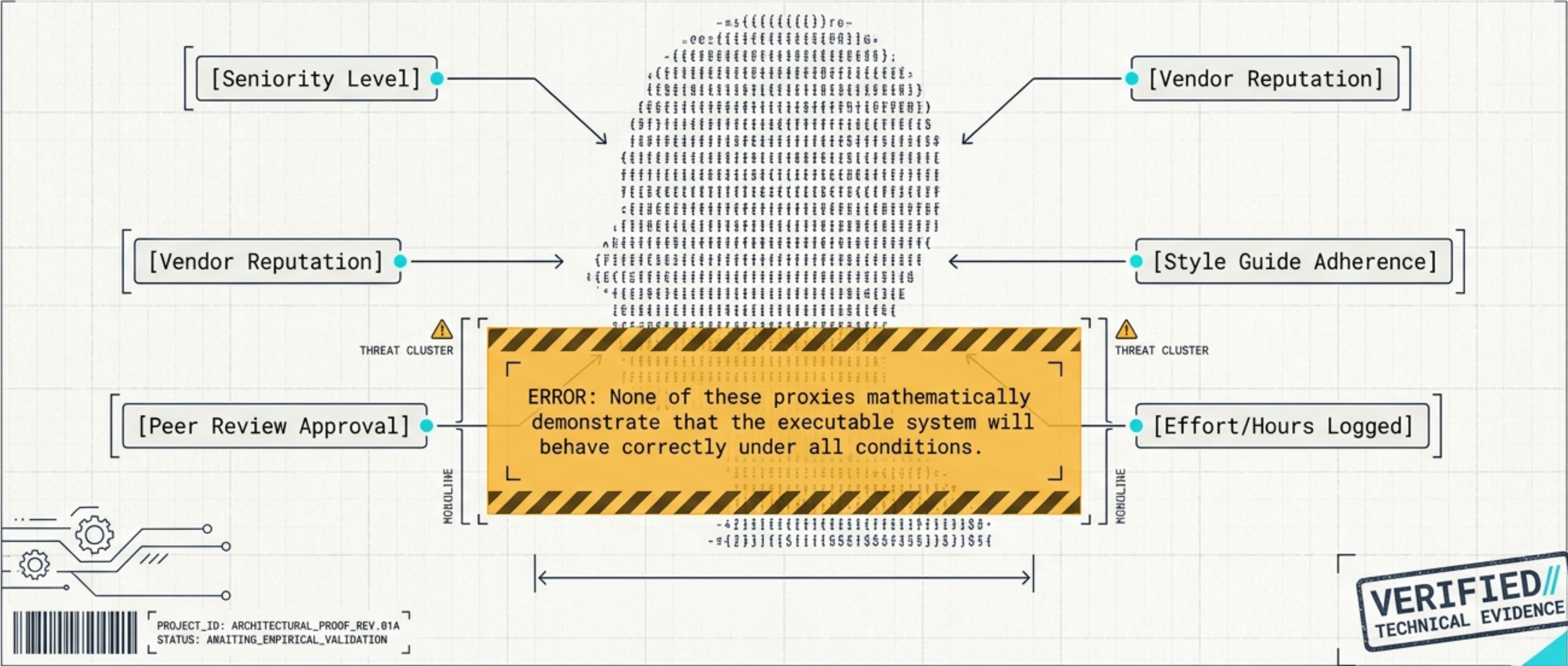
Governing Software Risk in the Age of Autonomous Intelligence



VERIFIED

The Proxies We Mistake for Proof

For decades, software trust has relied on circumstantial indicators of human reliability rather than empirical proof of system behavior.



The Human Fallibility Matrix

Human implementation introduces inevitable variability. The risk is not negligence; it is the biological and structural limits of human authorship.

01 // COGNITIVE & CAPACITY GAPS



- Cognitive variability & fatigue
- Scarcity, delay & vulnerability exposure
- Effort-based incentive misalignment

02 // TRANSLATION & ALIGNMENT GAPS



- Interpretation loss between purpose and code
- Unrecorded tacit knowledge & key-person dependency
- Subjective implementation choices

03 // SYSTEMIC RISK MULTIPLIERS



- Inconsistent security discipline (e.g., CISA/FBI 2024 warnings)
- Insider access & credential exposure
- Scaling individual decisions into systemic risk
- Human review as a weak substitute for evidence

Silicon Inherits Systemic Flaws

Delegating implementation to autonomous models introduces distinct, material vulnerabilities. Authorship by machine is no safer than authorship by human.

SECURITY REALITY



VERACODE 2025 GenAI Code Security Report:

- > 45% of assessed generated samples failed security tests

OWASP Agentic AI Risks Identified:

- > Goal hijacking
- > Identity abuse
- > Supply-chain compromise

PRODUCTIVITY REALITY



METR 2025 Randomized Study:

- > Experienced OS developers took 19% longer with early-2025 AI tools

GOOGLE DORA 2025 Finding:

- > AI amplifies existing organizational strengths AND weaknesses, rather than serving as a standalone solution.

The Three Paradigms of Assurance

	[1] Human-Centric (Status Quo)	[2] AI-Only (The False Promise)	[3] Evidence-Governed (The Blueprint)
Source of Trust	Developer Reputation	Model Capability	Independent Verifiability
Point of Failure	Fatigue & Tacit Knowledge	Hallucination & Prompt Injection	Flawed Requirements
Review Mechanism	Peer Pull Requests	Automated Self-Correction	Adversarial Testing & Invariants
Scalability Limit	Human Headcount	Automation Monoculture	Evidence Processing Capacity

VERIFIED

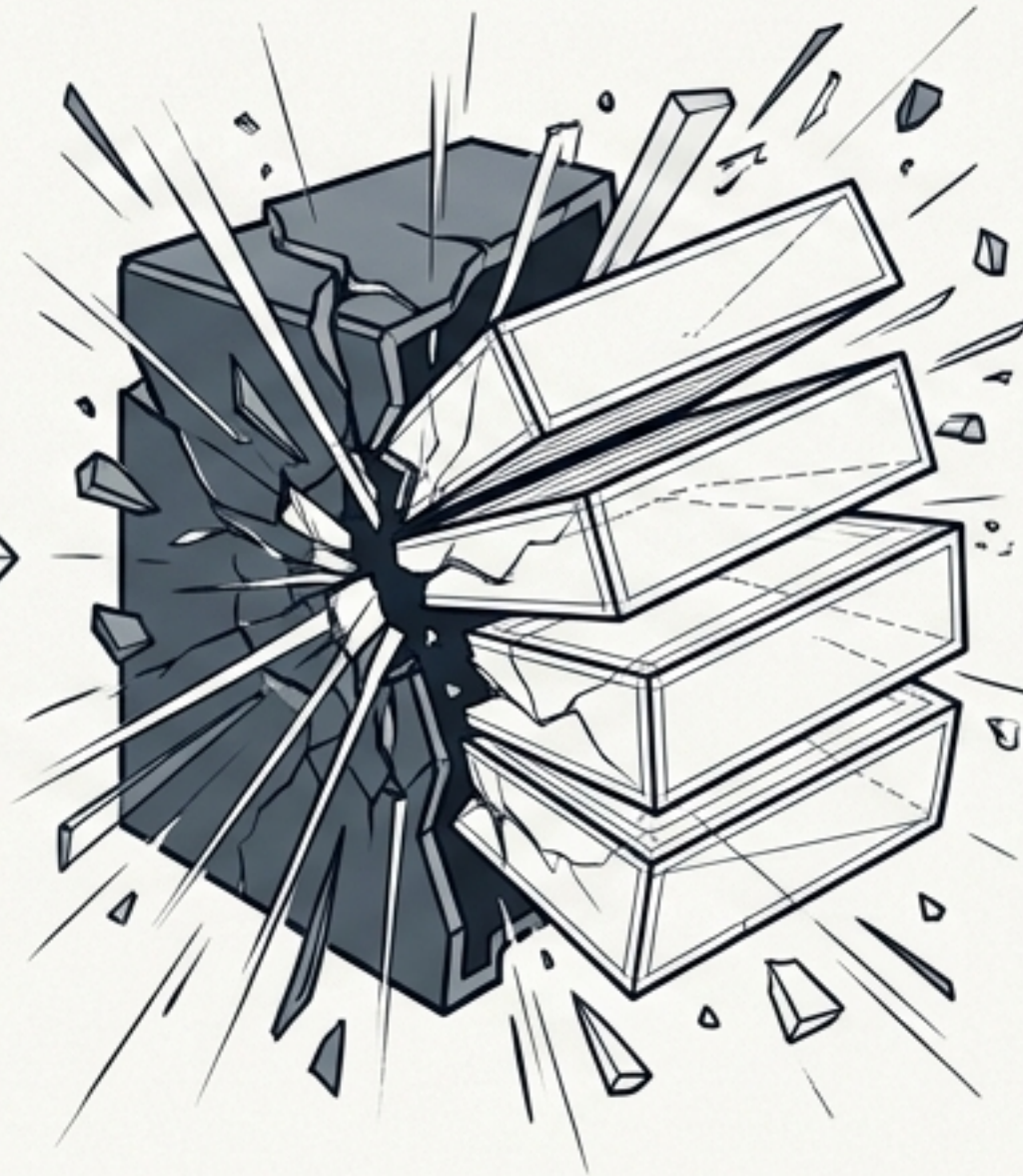
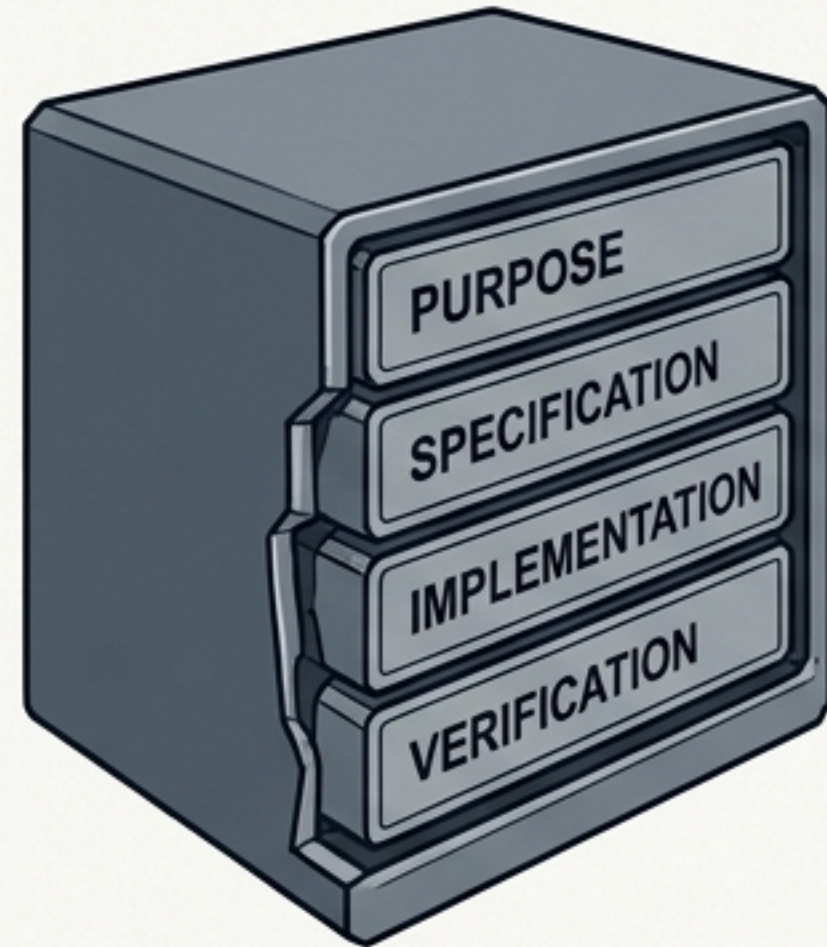
TARGET ARCHITECTURE

RISK HIGHLIGHT

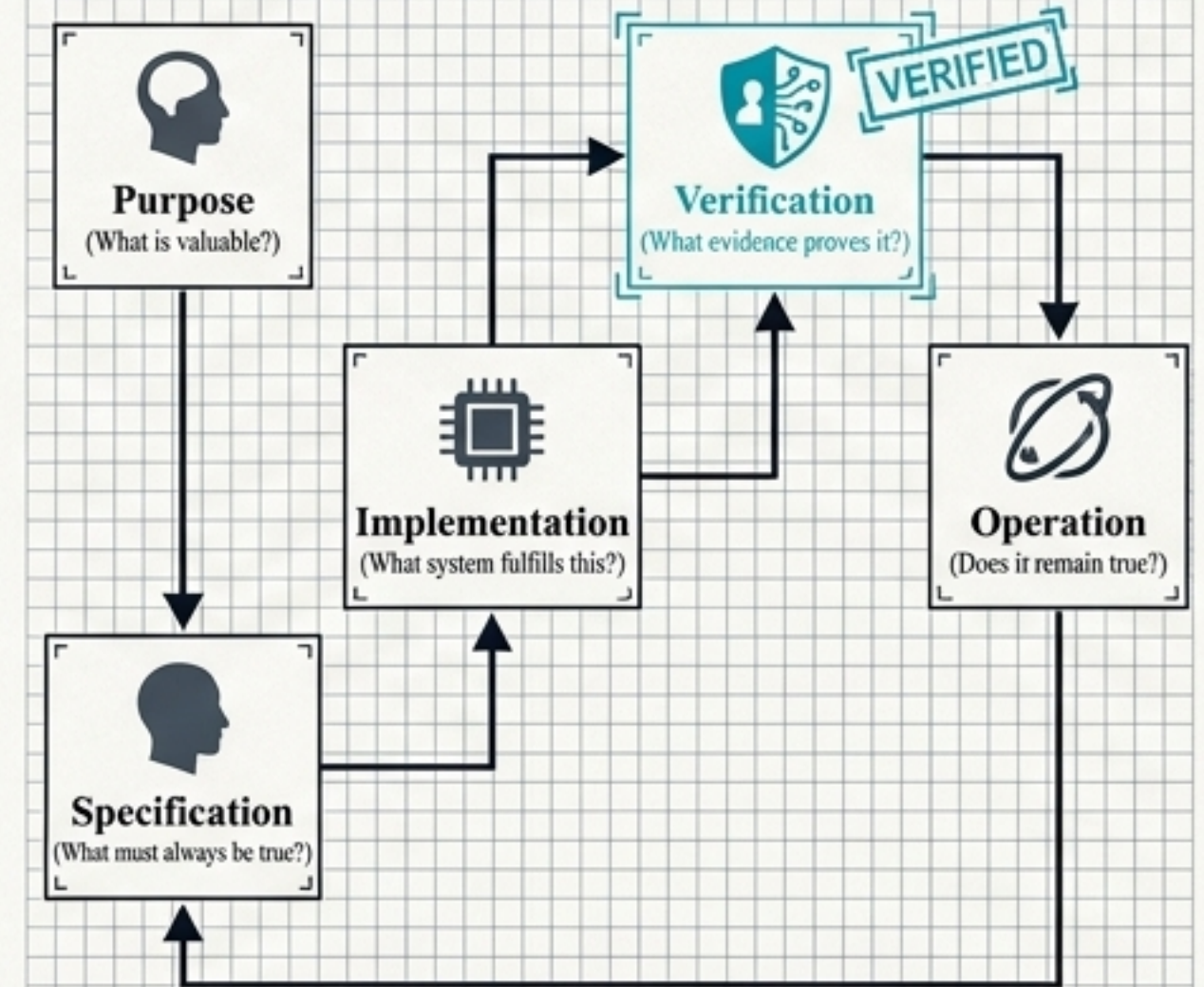
Unbundling the Developer Monolith

Autonomous engineering forces the separation of intent from implementation. Trust moves from the person who wrote the code to the system that verifies it.

[THE DEVELOPER]



[UNBUNDLED NETWORK]



The Executable Evidence Package

A mature autonomous system generates inspectable evidence, not just source code. Every change must carry its own manifest.

[ARTIFACT] Statement of Purpose	[ANSWERS] Why is this necessary?
[ARTIFACT] Statement of Purpose	[ANSWERS] What must be true?
[ARTIFACT] Acceptance Criteria & Invariants	[ANSWERS] What must be true?
[ARTIFACT] Threat Model & Constraints	[ANSWERS] What is prohibited?
[ARTIFACT] Implementation & Dependencies	[ANSWERS] What executes?
[ARTIFACT] Test & Verification Results	[ANSWERS] What proves safety?
[ARTIFACT] Provenance & Approvals	[ANSWERS] Who/what authorized it?
[ARTIFACT] Deployment & Rollback Plan	[ANSWERS] How is it contained?

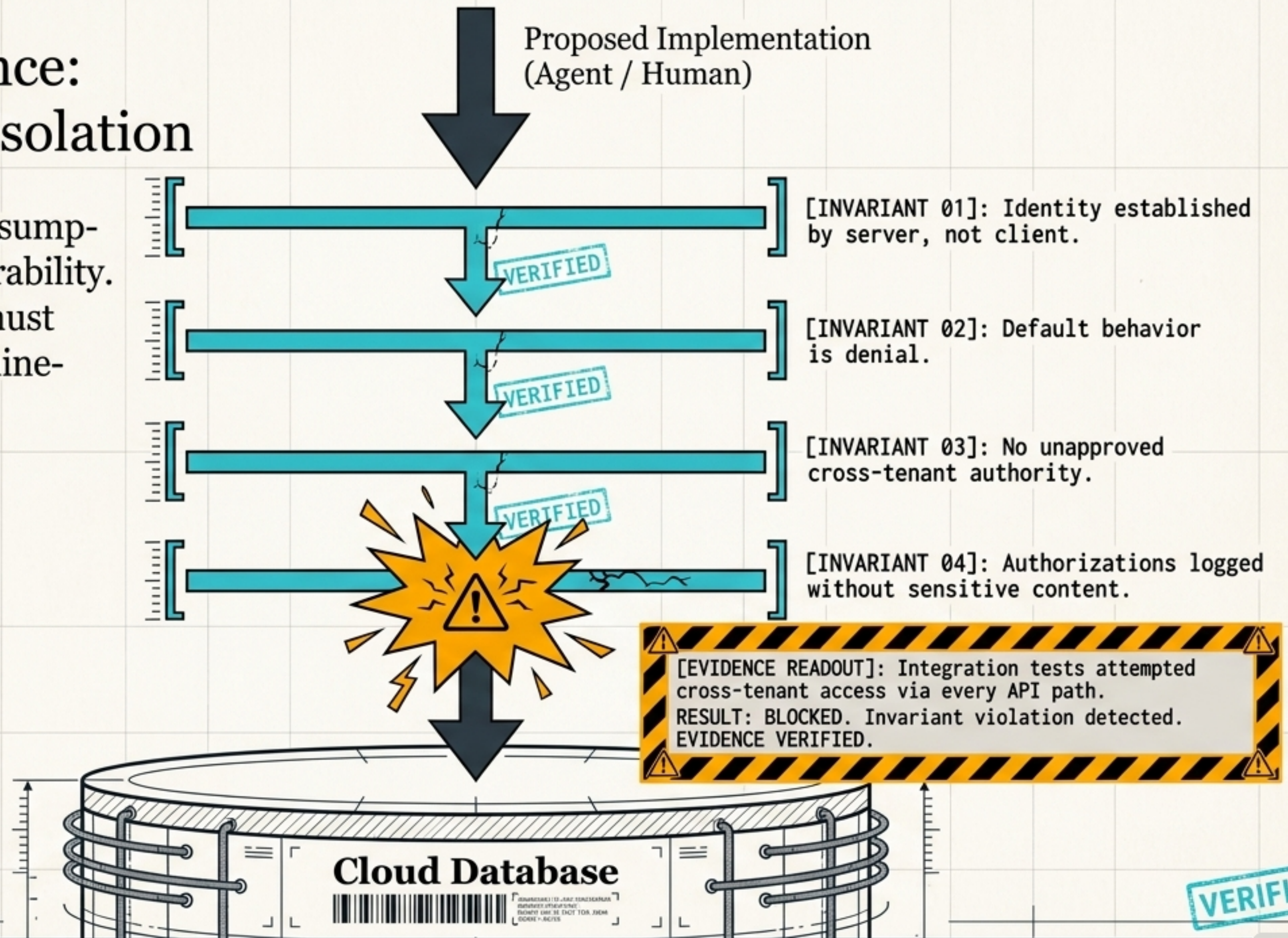


SHA-256: 8A4F...9B2

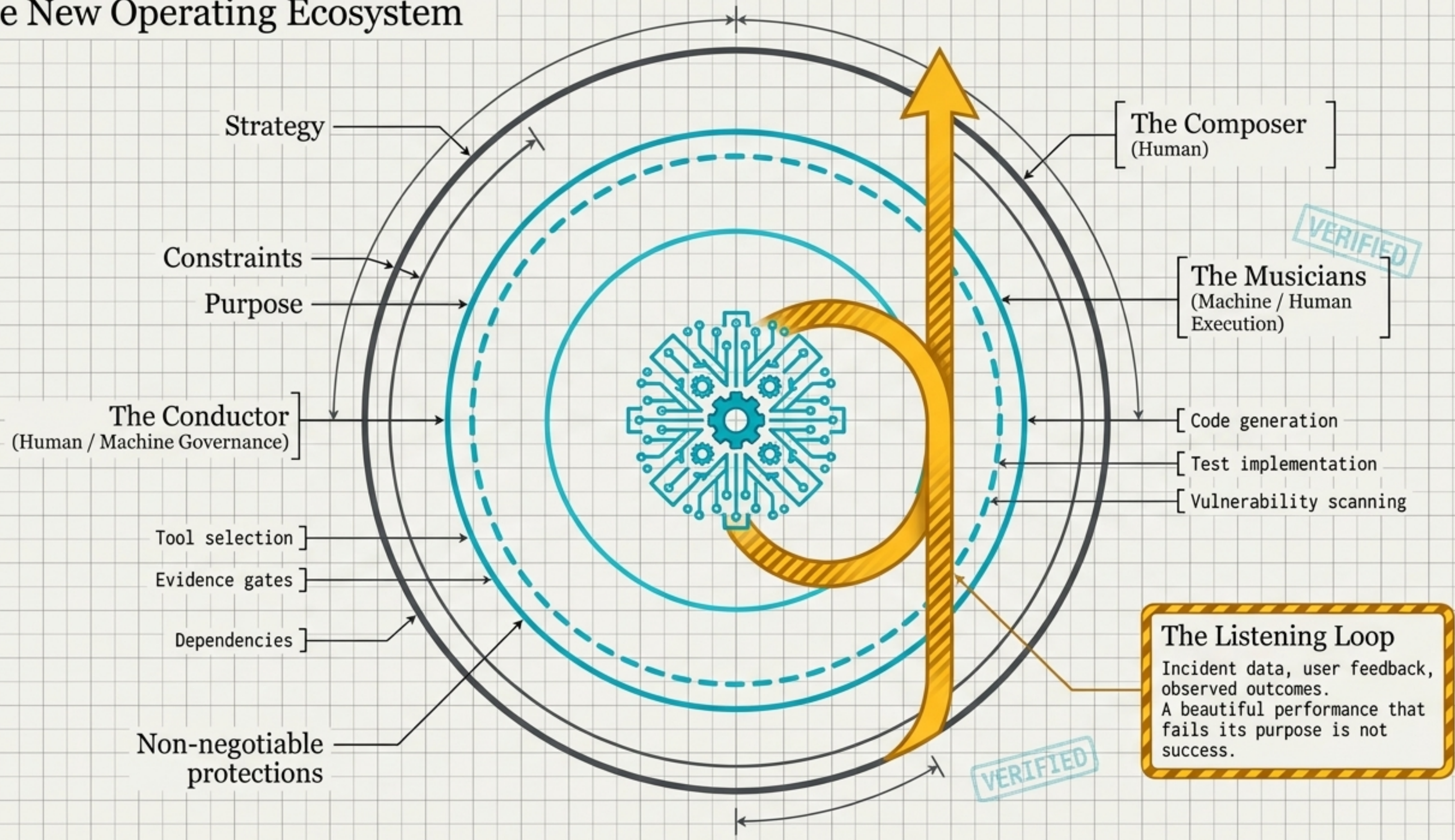


Applied Evidence: Multi-Tenant Isolation

Relying on unstated assumptions is a critical vulnerability. AI and humans alike must navigate explicit, machine-readable constraints.

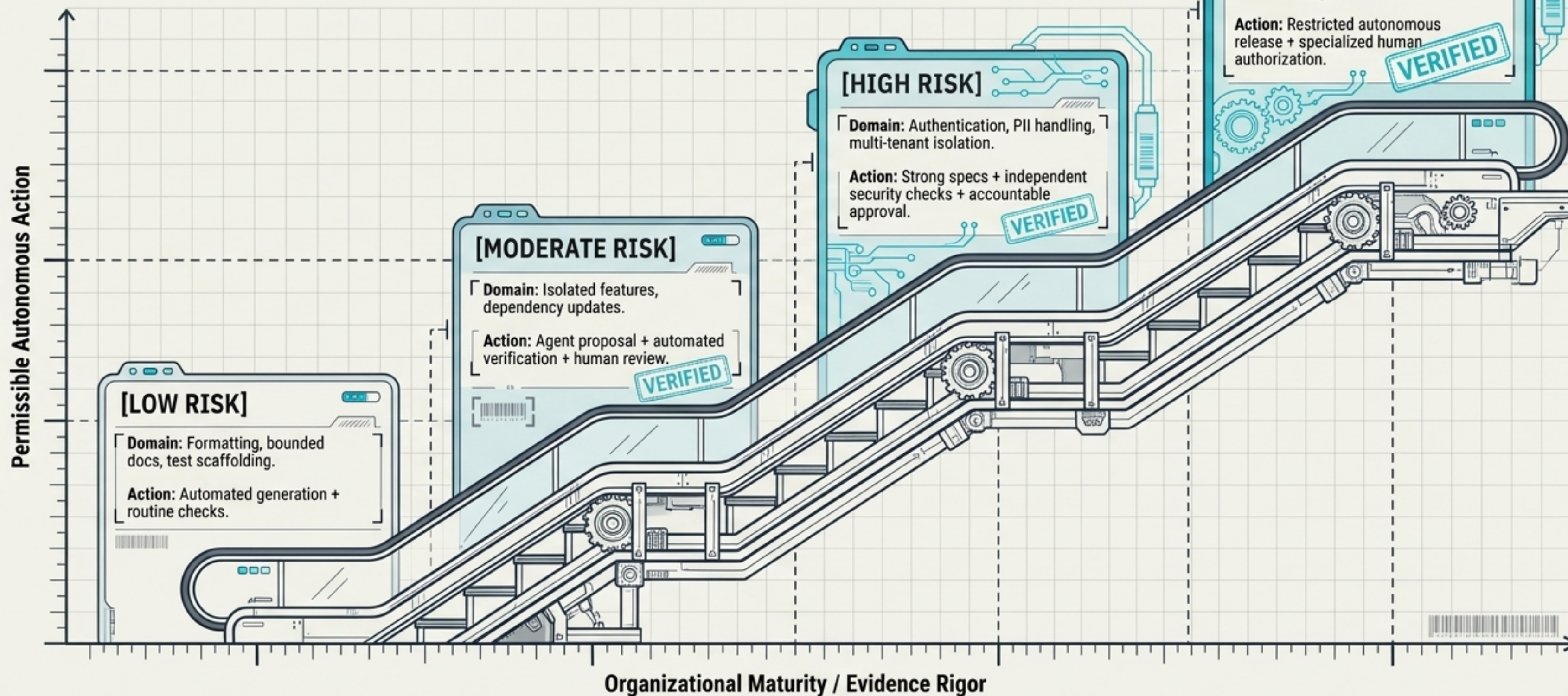


The New Operating Ecosystem



The Autonomy vs. Evidence Escalator

Organizations must not delegate all tasks at the same level.
Autonomy is earned through evidence rigor, not granted by default.



Measuring the Shift

Do not equate faster code production with better engineering.
Validate the hypothesis through operational outcomes.

[Escaped Defects]

severity-weighted
post-deploy issues



[Security Quality]

verified vulnerabilities
per change



[Change Failure Rate]

deployments
requiring
remediation



[Time to Verified Fix]

elapsed time: defect
to proven remediation



[Recovery Performance]

time to restore
service



[Specification Traceability]

requirements linked
to evidence



[Review & Rework Effort]

total human +
machine cost



[Operational Outcomes]

serving user value

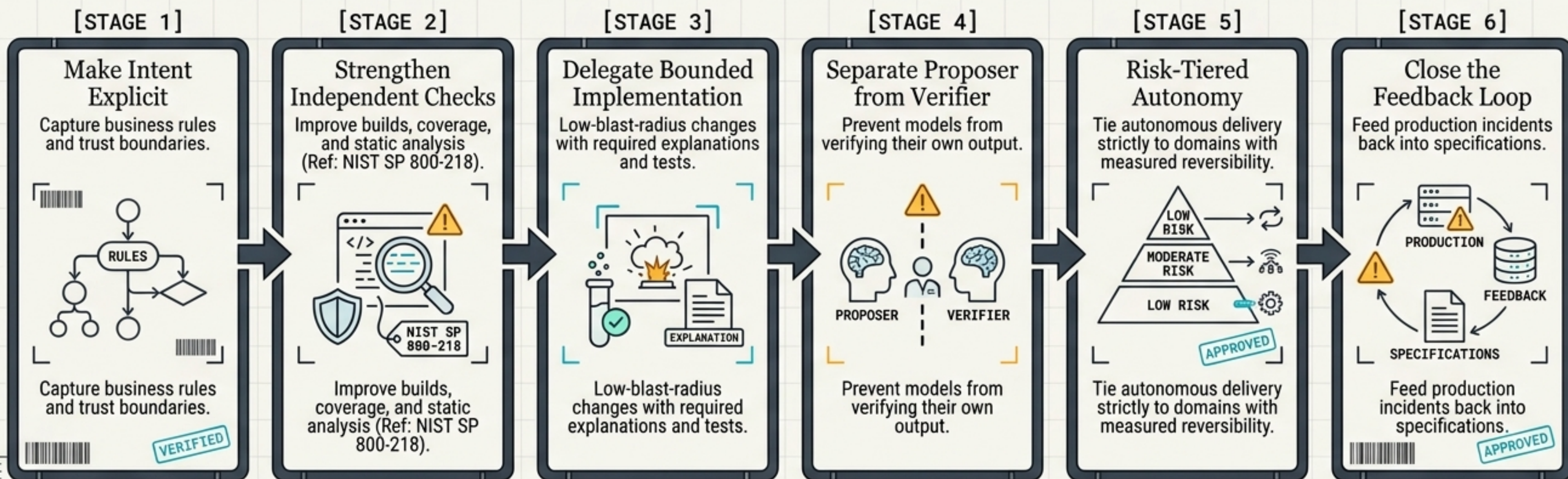


[Concentration Risk]

dependence on
single entity/model



Earning Autonomy: The Transition Roadmap

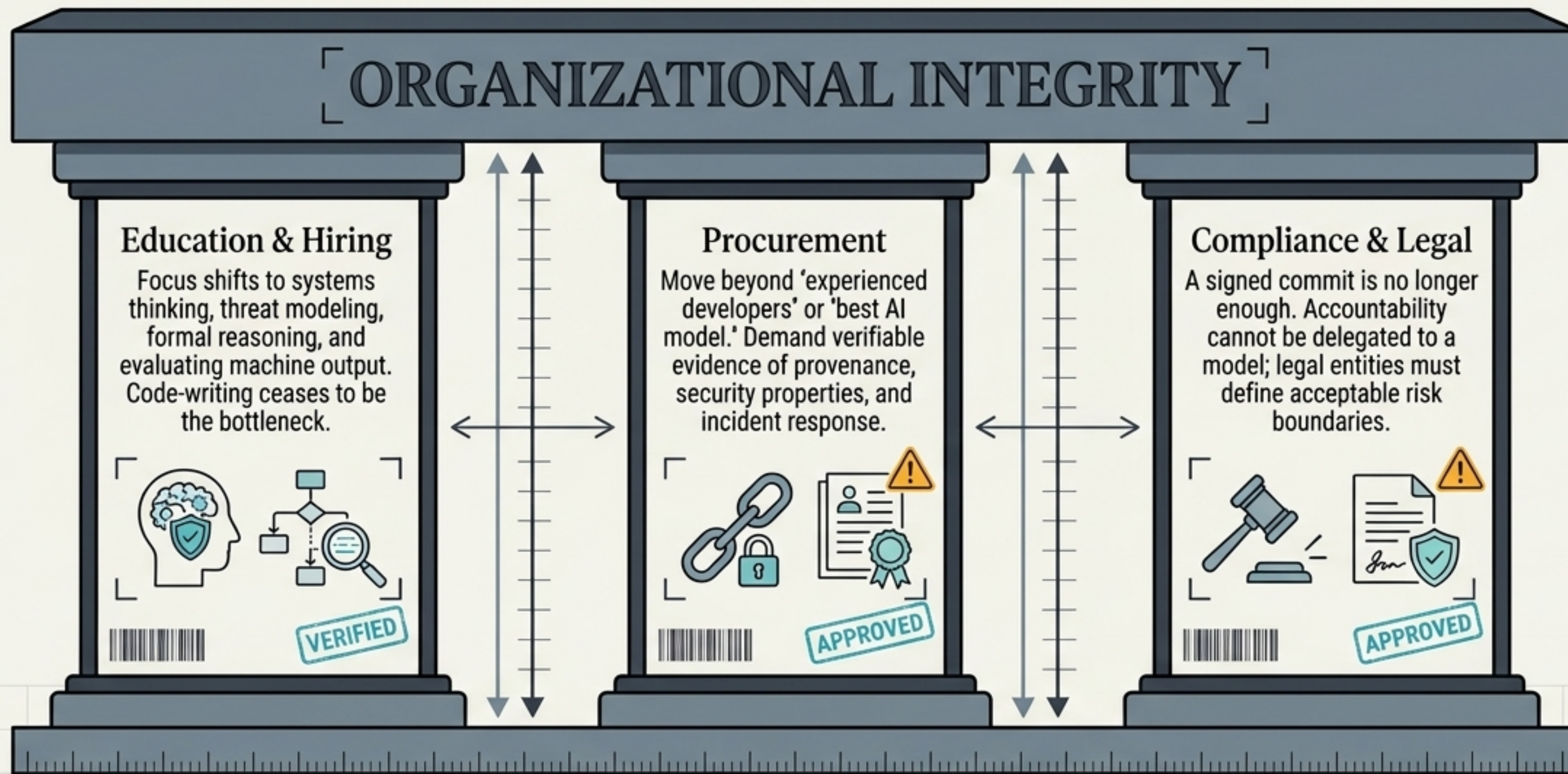


Iterative Reassessment Boundary

APPROVED

The Institutional Layer: Governance & Accountability

As code production becomes a commodity, the premium shifts to problem definition, threat modeling, and legal accountability.



CLASSIFICATION:
RESTRICTED

APPROVED

From Writing Code to Governing Behavior

```
function initializeSystem() {  
  const a = cause() {  
    consult.initialized();  
    this.stata = collectionSystem();  
    system.initializentent();  
  }  
  
  while(true) {  
    process(data);  
  }  
}
```

```
class UserVerifier {  
  constructor(db) {  
    this.db = db;  
  }  
}
```

```
// Legacy code block  
if(config.legacyMode) {  
  const { p = config.log(Mode)};  
  this.stata.legacyMode();  
} else {  
  // materas = obcested faotilciments
```

The guiding question is no longer simply, 'Who wrote this code?'
It is, 'What evidence justifies trusting this system to act?'

```
class UserVerifier {  
  constructor(db) {  
    this.db = db;  
    this.db.enPro  
  }  
}
```

```
// Legacy code block  
if(config.legacyMode) {  
  const m = mex;  
  if (!tredditeobrlou) {  
    this.db = egacyMate();  
    this.usexand = false;  
  } else {  
    this.db = exemontFatic();  
    this.userAtmExent();  
  }  
}
```

```
// Legacy code block {
```

```
{  
  amotedParamots);  
  tatorySystem();
```

```
er relice
```

```
lenalomait n;
```

```
// create testnet  
$nropset = mex;  
return $aropset;
```

```
function cons haffers() {  
  if(config.legacyMode) {  
    this.db = egacyMode("IP19");  
  } else {  
    this.data = dat;  
    this.evaltin();  
  }  
}
```

CLASSIFICATION:
RESTRICTED

[END_OF_DOSSIER_EVALUATION]